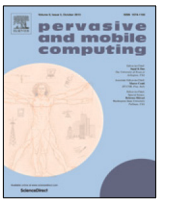
Pervasive and Mobile Computing 88 (2023) 101723



Contents lists available at ScienceDirect

Pervasive and Mobile Computing

journal homepage: www.elsevier.com/locate/pmc



Traffic aware dynamic load distribution in the Data Plane of SDN using Genetic Algorithm: A case study on NSF network



Chaitali Dey Bhowmik, Tirthankar Gayen *

School of Computer and Systems Sciences, Jawaharlal Nehru University, India

ARTICLE INFO

Article history:
Received 13 May 2022
Received in revised form 25 September 2022
Accepted 4 November 2022
Available online 21 November 2022

Keywords:
Software Defined Network
Load distribution
Latency
Genetic Algorithm

ABSTRACT

For a particular topology, with the best controller and switch positions SDN occasionally exhibits subpar performance, resulting in annoyance-inducing delays and unsatisfied users. Increased traffic volume can lead to congestion, which raises the burden on specific switches and controllers and lengthens the delay of data transmission. Additionally, the failure of specific nodes or links might result in an anomalous rise in the traffic load on specific switches and controllers. The conventional switch migration and congestion control techniques might not be sufficient to deliver the best throughput and capacity for frequent and unpredictable network traffic variations. Also, certain switches/controllers may experience heavy traffic load as compared to other switches/controllers. Under such adverse situations, the performance of the network becomes extremely poor. In light of these factors, this paper suggests an effective approach based on Genetic Algorithm for traffic load distribution in the Data Plane of SDN, which effectively reroutes packet flows (when such an adverse situation arises) in order to avoid abnormal load on switches/links and maintain acceptable network performance with the available resources. The proposed approach was applied to the NSF network and a case study was performed. The proposed approach was evaluated and found to be quite effective in reducing abnormal loads on switches/links as it is found that there can be a decrease in the average load by 396 units at the cost of increasing the average latency by 1.29 ms only.

© 2022 Elsevier B.V. All rights reserved.

1. Introduction

Software Defined Network (SDN) is a promising technology in today's networks and communication technology. The SDN architecture comprises of application plane, control plane and data plane where SDN application, SDN controllers and SDN switches are placed respectively. OpenFlow protocol is the driving communication protocol used in SDN. The SDN southbound interface maintains the connection between control plane and data plane whereas the northbound interface maintains the connection between the control plane and application plane. Controller is a crucial component of SDN which handles the overall network scenario. Multiple controllers are there to manage the distributed switches which act as packet forwarding entities. The switches consult with the controller for getting the forwarding rules from the lookup tables in controllers. One of the problems in SDN is the Controller Placement Problem (CPP) [1]. The proper positioning of controllers has a great impact on the performance, load balancing, reachability, or a combination of several factors in a communication network. The blast of mobile phone, virtualization of server, and cloud administration approaches

* Corresponding author.
E-mail address: tgayen77@gmail.com (T. Gayen).

https://doi.org/10.1016/j.pmcj.2022.101723
1574-1192/© 2022 Elsevier B.V. All rights reserved.

are some of the examples driving the framework organization industry to revaluate adjusted system framework models. Edgenet has put into place the Programmable Flow SDN Ecosystem, which creates Hyper-Converged storage/Converged storage using the Openflow controller. After acquiring Nicira in 2012, VMware developed the VMware SDN network virtualization and security platform, known as VMware NSX. In order to deliver lucrative, on-demand network services, the Nokia Network Services Platform (NSP) integrates service automation, network optimization, and dynamic assurance. It gives operators a more effective means to automate, optimize, and guarantee network services across numerous network layers, physical/virtual infrastructure, and gear from various manufacturers. Nuage Networks (a venture of Nokia), provides the Virtualized Services Framework (VSP) as a vendor-neutral platform for building clouds, from small private clouds to massive enterprise clouds. It is a SDN tool that connects compute resources as they are produced and virtualizes data centre network infrastructure. Various static and dynamic placement solutions have been proposed by considering different parameters like communication latency, controller load, controller utilization, control traffic overhead, resource consumption, flow set up time, QoS, etc. In most of the solutions the CPP is solved by clustering or by converting it to a multi-objective optimization problem which has been solved by heuristic or integer programming technique. Considering dynamic traffic load along with latency various researchers like Huque et al. [2], Bhowmik and Gayen [3] and others have presented approaches for dynamic controller placement. But, for a given topology, even with the optimal controller and switch positions, SDN is sometimes found to show poor performance causing delays resulting in annoyance and user dissatisfaction. One of the reasons for this subpar performance could be the increase in traffic load. The increase in traffic load due to the increase in certain type of communication among the hosts can cause congestion which can result in the increase in communication latency (delay). This may manifest as increase in load in the switches and controllers. Failure of certain nodes or links can cause abnormal increase in traffic load (excessive increase in the number of data packets, deviating from what is normal or usual, typically in a way that is undesirable or worrying) at certain switches and controllers resulting in the increase in latency (delay). The existing switch migration and congestion control techniques may not be adequate to achieve the best throughput and capacity due to the frequent and unexpected nature of network traffic variations. Additionally, compared to other switches and controllers, some of them may encounter a high traffic load. Under such adverse situations, the performance of the network becomes extremely poor resulting in irritating delays (delays resulting in displeasure or annoyance) and dissatisfaction among the users of the network. The existing techniques of load distribution are not always found to be efficient since they either use switch migration techniques, focus on load balancing in the data plane using the traditional Shortest Path approach, or focus on load balancing in the Control Plane of SDN. Considering these aspects, an effective approach using traffic aware artificially intelligent dynamic load distribution in the Data Plane has been proposed by re-routing the packet flows (when such adverse situation arises) in an effective way so that an acceptable performance of the network can be maintained with the existing resources.

The contributions of importance in this article are: 1. This article presents an approach using traffic-aware artificially intelligent dynamic load distribution in the Data Plane of SDN by re-routing the packet flows (when adverse situation arises) in an effective way so that an acceptable performance of the network can be maintained with the existing resources. 2. The proposed approach was applied to the NSF network and a case study was performed. 3. The proposed approach was evaluated and found to be quite effective in reducing abnormal loads on switches/links as it is found that there can be a decrease in the average load by 396 units at the cost of increasing the average latency by 1.29 ms only. 4. Further, a regression analysis using the load variance values is presented to confirm that the proposed approach is more effective in providing less load variance as compared to the traditional Shortest Path (SP) algorithm. The rest of the article is organized as follows: in Section 2 the related work is provided; Section 3 describes the proposed Traffic Aware Dynamic Load Distribution; Section 4 describes the proposed TAFR and GAEFP algorithms; Section 5 shows the case study based on the implementation of the proposed algorithms on NSF network; Section 6 shows the evaluation of the proposed approach; Section 7 contains the conclusion with the scope for future work.

2. Related work

To estimate the real-time traffic for requests that are added to the network in passive mode, Tahaei et al. [4] have proposed a mechanism where increased overheads associated with controller computation and communication cost was minimized while maintaining accuracy. To improve network performance, Chien et al. [5] proposed a load balancing framework for service-oriented SDN-SFC where classification of the request was based on the type, and to each service a priority was assigned and further used a heuristic algorithm to decide on taking a specific transmission path among the service function chains available. Although, their technique improved the degree of load balancing and shortened data transmission time, various other parameters (like execution time, size of the task, response time, etc.) which can affect the performance of load balancing were hardly given importance. Ejaz et al. [6] used controller duplication and migration for load balancing using Virtual SDN (vSDN) to reduce delay and improve throughput & bandwidth utilization. But, it increased complexity with the addition of additional controllers. Also, it resulted in high energy consumption and carbon emission. Xu et al. [7] used switch migration technique for load balancing serial and non-serial switch requests but the assumptions made on path computation, and cost of installing flow entry may not be realistic. Jin et al. [8] made improvement in throughput using tenant throughput allocation but did not pay attention to latency and reliability. Gao et al. [9] improved the performance packet transmission by using hierarchical geographical routing protocol but it increased overhead lacked along with lack of accuracy and communication between nodes. Hu et al. [10] used all pair

shortest path but it resulted in overloaded switches and controllers resulting in delay and increased computation cost. Lan et al. [11] used hierarchical control plane resulting in few switch migrations but it was topology dependent and it resulted in under utilization of resources. Gao et al. [12] used centralized algorithm based on max utility function in LTE-V networks to improve the uplink and downlink rate but had fixed threshold. Also, it did not consider performance metrics. Koryachko et al. [13], Wang et al. [14], Lin et al. [15], Chen et al. [16], Raza et al. [17] used shortest path routes from source to destination for the optimal the use of resources and improve the QoS performance but failed to address the problem where certain switches/controllers may experience heavy traffic load as compared to other switches/controllers in some situations. Attarha et al. [18] used re-routing algorithm for congestion control in order to achieve the best backup path but it depends on the transmission rate and requires maintenance of the backup paths. Chen et al. [19] used adaptive network connection to improve network connection but it is static and cost ineffective. Zhao et al. [20] suggested the use of multiple controllers to reduce the latency but it is not scalable. Also, it failed to address the problem where certain switches/controllers may experience heavy traffic load as compared to other switches/controllers in some situations. The multi-controller deployment algorithm proposed in Jiugen et al. [21], Ma et al. [22], Wang et al. [14] balances the load among the controllers placed in different regions but not among the switches. Re-routing the traffic is a solution, where Hu et al. [10], Attarha et al. [23], Hwang et al. [24], Ramdhani et al. [25], Boero et al. [26], Chakravarthy and Amutha [27] have used and modified the flow rules to improve the degree of load balancing but with increasing overhead and delay. Priyadarsini et al. [28] considered self-adaptive load balancing scheme (without considering delay) to minimize the switch migration but with the increase in cost and delay. Rangisetti et al. [29] reduced overload in LTE networks without considering delay and jitter. Al-Tam and Correia [30] proposed a heuristic approach for switch migration to balance the load using optimal controller selection but it is centralized, suffered from single point failure, migration cost has not been evaluated and bandwidth is underutilized. Zhang et al. [31] focussed on balancing the load by reducing the average response time of the controller but have not considered the requests from diverse flow and other QoS metrics. Pietrabissa et al. [32] used game theory to avoid single point failure but increased SDN proxies. Farhoud et al. [33] addresses server load balancing using heuristic algorithm with optimization but did not take into consideration the network performance. Wang et al. [34] used greedy method and did switch migration considering the tradeoff between migration cost and load balance rate but it lead to increased overhead and lack of security. Mahlab et al. [35] tried for optimal load distribution using Genetic Algorithm but it resulted in local optima. Yu et al. [36] used Deep Neural Network with backpropagation and tried for load balancing with efficient path selection but it used limited parameters and resulted in increased overhead. Cimorelli et al. [37] used Game Theory and Wardrop Equilibrium for load balancing across multiple controllers but it resulted in local minimum and cluster overhead. He et al. [38] used modified Particle Swarm Optimization for achieving balancing the load in the controllers but it resulted in increased energy consumption and lack of security. Liao et al. [39] used binary tree for load balancing by adjusting flow table rules did not use QoS metrics. It is complex and topology dependent. Sathyanarayana and Moh [40] used Ant Colony Optimization, server-load and network statistics to the find the best server and the best path for network flows but with increasing overhead. Yang et al. [41] used Neural network and K-means for better controller selection but it used inaccurate prediction with high energy consumption. Manzoor et al. [42] used Support Vector Machine and Fuzzy based Particle Swarm Optimization for multi-controller load balancing but it was not dynamic, leads to increase in overhead and it categorized flow based on priority. Xue et al. [43] used Genetic-Ant Colony Optimization for reducing transmission time and packet loss but considered few parameters. It also resulted in increase in overhead. Rofie et al. [44] used Dijkstra's shortest path algorithm for Openflow based load balancing it failed to address the problem where certain switches/controllers may experience heavy traffic load as compared to other switches/controllers in some situations. Maity et al. [45] proposed a load reduction strategy that prioritizes traffic flows based on QoS demands and aims to avoid link congestion and rule-space overflow during flow migration but it resulted in increase of overhead and latency.

Failure of certain nodes or links can cause abnormal increase in traffic load at certain switches and controllers resulting in the increase in latency (delay). The existing switch migration and congestion control techniques may not be adequate to achieve the best throughput and capacity due to the frequent and unexpected nature of network traffic variations. Additionally, compared to other switches and controllers, some of them may encounter a high traffic load. Under such adverse situations, the performance of the network becomes extremely poor resulting in irritating delays and dissatisfaction of the users of the network. Thus, there is a need for multi-objective optimization for dynamic load distribution in the Data Plane of SDN order to minimize the load variance values among the switches keeping the communication latency of the flows (from source to destination) as less as possible. However, it is essential to identify a simple and practical multi-objective optimization technique in this situation which involves less overhead and low communication latency. According to Nair and Sooda [46], in the case of the Genetic Algorithm, the probability of shortest path convergence increases with the number of iterations, whereas in the case of the Simulated Annealing, the number of iterations was found to have almost no influence on obtaining better results because it operates on the random principle of selection. Genetic algorithms are known to search in parallel from a population of points. Therefore, unlike conventional approaches that search from a single point, it has the potential to avoid becoming locked in local optimal solution. On the other hand, Particle Swarm Optimization's (PSO's) local search space lacks diversity, making it easy to fall into a local optimum in a high-dimensional space. Although, Gravitational Search Algorithm (GSA) produces some outcomes that are adequate, it is prone to diverge and become trapped in the local optima [47]. Thus, Genetic Algorithm is found to be more effective. In light of these factors, this article suggests an effective, intelligent and multiobjective optimization based on Genetic Algorithm for dynamic load distribution in order to minimize the load variance values among the switches keeping the latency of the flows (from source to destination) as less as possible.

3. Traffic aware dynamic load distribution

With the existing load values of the switches, the master controller computes the effective flow paths from all the sources to their destinations for the movement of data packets, after a fixed interval of time (as decided by the network administrator). When adverse situation arises, the master controller updates the flow table entries in the switches through the slave controller based on the flow paths computed by the master controller.

$\neg_{ij}$ represent the average number of flows over the current time slot originating from the switch S_i to switch S_j.

$X_{ik} = 1$, if switch S_i is connected to the controller C_k

$= 0$, otherwise

$\neg_{ii}X_{ik}$ represents the average number of flows (from the switch S_i to the controller C_k and from the controller C_k to the switch S_i) over the current time slot [45]. The intermediate path setup requests at switch S_i for the flows coming from a neighbour switch controlled by a different controller $= \sum_{\forall S_j} \neg_{ji}X_{jk'}(1 - X_{ik'})$, where i ≠ j. Any controller other than C_k is denoted by $C_{k'}$. The rule installation load at each switch controlled by C_k for all flows traversing the domain controlled by $C_k = \sum_{\forall S_j}(\neg_{ji}+\neg_{ij})X_{ik}$. Total load in the switch S_i

$$L(S_i) = \neg_{ii}X_{ik} + \sum_{\forall S_j} \neg_{ji}X_{jk'}(1 - X_{ik'}) + \sum_{\forall S_j}(\neg_{ji} + \neg_{ij})X_{ik} \tag{1}$$

Let L(S) represent the load on switch S. Considering a constant value for the rule installation load, the load in the switch S_i, $L(S_i)$ over a given period of time is directly proportional to the number of flows in which switch S_i is used considering all the flows from source to destination over a given period of time. For example, considering a scenario with the following three flows:

F_{12} : 1 4 2

F_{28} : 2 4 5 8

F_{38} : 3 2 1 4 5 8

Flow F_{12} uses the sequence of switches 1 4 2, flow F_{28} uses the sequence of switches 2 4 5 8 and flow F_{38} uses the sequence of switches 3 2 1 4 5 8. Here, switch 1 is used in 2 flows, switch 2 is used in 3 flows, switch 3 is used in 1 flow, switch 4 is used in 3 flows, switch 5 is used in 2 flows, switch 8 is used in 2 flows. Considering the abovementioned flows (F_{12}, F_{28}, F_{38}) the following relations using various switch loads can be specified:

$L(S_3) < L(S_1) < L(S_2)$, $L(S_3) < L(S_5) < L(S_4)$, $L(S_3) < L(S_8) < L(S_4)$, $L(S_3) < L(S_1) < L(S_4)$, $L(S_3) < L(S_5) < L(S_2)$, $L(S_3) < L(S_8) < L(S_2)$

The average load on the switches in the domain controlled by controller C_k is given as

$$\mu_k = \frac{\sum_{\forall S_j} L(S_i)}{\sum i} \tag{2}$$

The variance

$$\sigma_k^2 = \frac{\sum_{\forall S_j}(L(S_i) - \mu_k)^2}{\sum i} \tag{3}$$

Let Lat(S) represent the latency due do switch load on S.

$$Lat(S) \propto L(S)$$

$$\text{or , } Lat(S) = \beta L(S) \tag{4}$$

where, β is the constant switch-latency factor, which is defined as the latency per unit load of the switch S.

For heterogeneous switches, the switch-latency factor β can be considered for every type of switch. Thus, the latency in $path_t$, $Lat(path_t)$, is proportional to the sum of the latency values due to switch load (for all switches in the path t) and latency values of all the edges/links in path t.

Mathematically,

$$Lat(path_t) = \sum_{\forall S_i \ in \ path_t} Lat(S_i) + \sum_{\forall E(i,j) \ in \ path_t} Lat(E(i,j)) \tag{5}$$

$$or, \ Lat(path_t) = \sum_{\forall S_i \ in \ path_t} \beta_i L(S_i) + \sum_{\forall Z(i,j) \ in \ path_t} Z(i,j) \tag{6}$$

where, β_i is the constant switch-latency factor, which is defined as the latency per unit load of the switch S_i

$L(S_i)$ represents the load of the switch S_i

$E(i,j)$ represents the link between the switch S_i and switch S_j

$Lat(E(i,j))$ represents the latency in the link between the switch S_i and switch S_j

$Z(i,j)$ represents the latency in the link between the switch S_i and switch S_j

Thus, the objective is to find a good solution for load distribution in order to minimize the value of σ_k^2, $\forall S_i$ in the domain controlled by the controller C_k, such that the latency of the flows from source to destination is as less as possible. Table 1 provides descriptions of various notations used in this article.

Table 1
Notations and their descriptions.

Notation	Description
F_{ij}	It represents flow from switch i to switch j
β_i	It is the constant switch-latency factor, which is defined as the latency per unit load of the switch S_i
$L(S_i)$	It represents the load of the switch S_i
$E(i,j)$	It represents the link between the switch S_i and switch S_j
$Lat(E(i,j))$	It represents the latency in the link between the switch S_i and switch S_j
$Z(i,j)$	It represents the latency in the link between the switch S_i and switch S_j
T	Traffic matrix
SC	Switch-to-controller connection matrix
Z	Switch-to-switch link latency matrix
F	Set of all flow paths from the source node to the destination node
Lat_s	Shortest path latency
μ_k	Average load on the switches in the domain controlled by controller C_k
σ_k^2	Load variance on the switches in the domain controlled by controller C_k
V_s	Load variance using Shortest Path

4. Proposed approach

In order to find the effective flow path to minimize the value of σ_k^2($\forall S_i$ in the domain controlled by the controller C_k) with the minimum latency value from source to destination an approach is developed using the proposed algorithms TAFR and GAEFP . Using algorithm GAEFP an effective and intelligent traffic aware flow routing algorithm TAFR has been developed for dynamic load distribution in the data plane of SDN. The following are the steps of the algorithm TAFR.

Algorithm: Traffic Aware Flow Routing (TAFR)

Require: Traffic matrix, T
Switch-to-controller connection matrix, SC
Switch-to-switch link latency matrix, Z
Set of all flow paths from the source node to the destination node, F
Threshold, Th

Ensure: Set of effective flow paths, $F_{effective}$

1: Repeat after every time interval tp {

1.1: **Initialize** Traffic matrix (**T**), Switch-to-controller connection matrix (**SC**), Switch-to-switch link latency matrix (**Z**), Set of all pairs of source and destination nodes (**SD**), Threshold (**Th**) in the master controller
$Lat_{sh} \leftarrow 0$, F$\leftarrow \Phi$, sh_path_lat$\leftarrow \Phi$;

1.1.1: $\mu_k \leftarrow \frac{\sum_{\forall S_j} L(S_i)}{\sum i}$

1.1.2: $\sigma_k^2 \leftarrow \frac{\sum_{\forall S_j} (L(S_i)-\mu_k)^2}{\sum i}$

1.2: **for** every switch S_i (1 <= i <= m) in the domain controlled by every slave controller C_k (1 <= k <= n) the master controller computes

1.2.1: **for** f $\leftarrow$ 1 to flows

1.2.1.1 $F_{sd} \leftarrow \Phi$

1.2.1.2: Insert the set of all possible paths for the flow f from the source node to the destination node in the set F_{sd}

1.2.1.3: Insert F_{sd} in the set F

1.2.1.4:

$$m_lat_{ft} \leftarrow \sum_{\forall S_i in\ path_t} \beta_i L(S_i) + \sum_{\forall Z(i,j)\ in\ path_t} Z(i,j)$$

//m_lat_{ft} represents the minimum latency value for the flow f
// $Z(i,j)$ represents the latency in the link between the switch S_i and switch S_j for the flow f and path t
// 1 <= t <= number of paths for the flow f
// β_i is the constant switch-latency factor, which is defined as the latency per unit load of the switch S_i

1.2.1.5:$sh_path_f \leftarrow \underbrace{argmin}_{t}(m_lat_{ft})$

//sh_path_f represents the shortest path (path having minimum latency) for the flow f

1.2.1.6: $m_lat_f \leftarrow m_lat_{sh_{path_f}}$
// $m_lat_f$represents the minimum latency value for the flow f

1.2.1.7: Insert the path m_lat_f into the set sh_path_lat

1.3: **end for**

1.4:

$$Lat_{sh} \leftarrow \frac{\sum_{\forall f \in sh_path_lat} m_lat_f}{\sum f_{\forall f \in sh_path_lat}}$$

// Lat_{sh} represents the average latency value for the shortest path

1.5: $\mu_s \leftarrow \frac{\sum_{\forall S_i} L(S_i)}{\sum i}$

1.6: $V_s \leftarrow \frac{\sum_{\forall S_i}(L(S_i)-\mu_s)^2}{\sum i}$ // V_s represents the average load variance for shortest path

1.7: $F_{effective} \leftarrow GAEFP(T, Z, Lat_{sh}, F, V_s)$;

1.8: **for** every switch S_i (1 <= i <= m) and every slave controller C_k (1 <= k <= n) the master controller computes

1.8.1: **if** ($L(S_i) > Th$)

1.8.1.1: Update the flow tables of each switch through the corresponding slave controller based on $F_{effective}$

1.8.1.2: $F \leftarrow F_{effective}$

1.8.1.3: break

1.8.2: **end if**

1.9: **end for**

} // end repeat

The switch-to-switch latency matrix, Z (an example shown in Fig. 2) specifies the latency values of the link between any two switches of the network. For switches which are not directly connected by a link the latency value is considered to be infinitely high (represented as E) (see Table 2).

The latency value of each path t (excluding the switch latency) from source to destination (where $1 \leq t \leq$ number of paths for the flow $\boldsymbol{f}$) for the flow $\boldsymbol{f}$ is $Lat(path_{ft})$ where

$$Lat(path_{ft}) = \sum_{\forall Z(i,j)\ in\ path_{ft}} Z(i,j) \tag{7}$$

In the algorithm **TAFR**, after every interval of time t_p, the master controller initializes the Traffic Matrix $\boldsymbol{T}$, Switch-to-controller connection matrix $\boldsymbol{SC}$, Switch-to-switch link latency matrix $\boldsymbol{Z}$, Set of all pairs of source and destination nodes

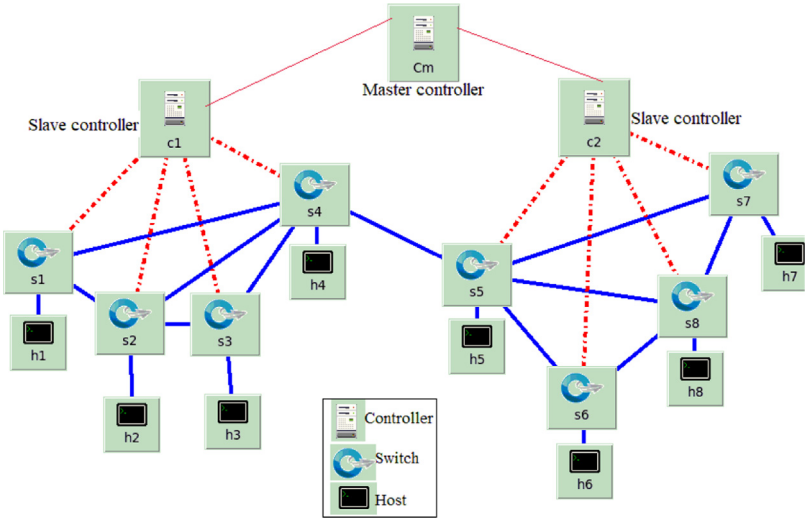


Fig. 1. An example SDN with master–slave controllers.

Table 2
Switch-to-switch latency matrix, Z of Fig. 1.

Z	1	2	3	4	5	6	7	8
1	*E*	*0.001*	*E*	*0.002*	*E*	*E*	*E*	*E*
2	*0.001*	*E*	*0.001*	*0.001*	*E*	*E*	*E*	*E*
3	*E*	*0.001*	*E*	*0.001*	*E*	*E*	*E*	*E*
4	*0.001*	*0.001*	*0.001*	*E*	*0.001*	*E*	*E*	*E*
5	*E*	*E*	*E*	*0.001*	*E*	*0.001*	*0.002*	*0.002*
6	*E*	*E*	*E*	*E*	*0.001*	*E*	*E*	*0.001*
7	*E*	*E*	*E*	*E*	*0.002*	*E*	*E*	*0.001*
8	*E*	*E*	*E*	*E*	*0.002*	*0.001*	*0.001*	*E*

SD, Threshold, ***Th***. For every switch S_i ($1 \leq i \leq m$) in the domain controlled by every slave controller C_k ($1 \leq k \leq n$) the master controller computes $Lat(S_i) = \beta_i L(S_i)$. It also computes μ_k, σ_k^2, and $F_{effective}$ using the algorithm ***GAEFP***. Then, for every switch S_i ($1 \leq i \leq m$) and every slave controller C_k ($1 \leq k \leq n$) the master controller checks whether $L(S_i) > Th$. If $L(S_i) > Th$, then update the flow tables of each switch through the corresponding slave controller based on $F_{effective}$ data and update $\boldsymbol{F}$ to $F_{effective}$. Algorithm ***GAEFP*** uses genetic algorithm to find an effective traffic aware flow route for dynamic load distribution in the data plane of SDN. It uses the standard Tournament Selection which randomly samples $\boldsymbol{k}$ individuals with replacement from the current population of size $\boldsymbol{P}$ into a tournament of size $\boldsymbol{k}$ and selects the one with the best fitness from the tournament. According to Fang and Li [48] Tournament selection has a selection pressure can be adjusted easily, does not require sorting the whole population first and has the time complexity ***O(P)***. According to Lavinas et al. [49] and Blickle et al. [50] commonly used tournament sizes are 2, 4 and 7. Higher values for the tournament size increases the selection intensity. The following are the steps of the algorithm ***GAEFP***.

Algorithm: Genetic Algorithm based Effective Flow Path (GAEFP)

Require: Traffic matrix, T
Switch-to-switch link latency matrix, Z
Shortest path latency, Lat_s
All paths flow matrix, F
Load variance using Shortest Path, V_s

Ensure: Set of effective flow paths, $F_{effective}$

1: **Initialize** Population size N_p, nEliteKids N_E, CrossoverFraction F, NGenerations N_G

2: Using the all paths flow matrix, F, generate N_p individuals such that every individual is formed by randomly selecting one path (from the set of all possible paths between the pair of hosts for each flow) and concatenating them.

3: Insert all the N_p individuals (created in step 2) into the initial set **P**

4: **for** i $\leftarrow$ 1 to N_G

4.1: **for** each **j** individual in **P**

4.1.1: $\mu_j \leftarrow \frac{\sum_{\forall S_j} L(S_i)}{\sum i}$

4.1.2: $\sigma_j^2 \leftarrow \frac{\sum_{\forall S_j} (L(S_i) - \mu_j)^2}{\sum i}$

4.1.3:

$$Lat(path_t) \leftarrow \sum_{\forall S_i in\ path_t} \beta_i L(S_i) + \sum_{\forall Z(i,j)\ in\ path_t} Z(i,j)$$

4.1.4: $Lat(j) \leftarrow 0$

4.1.5: **for** each path **t** of the individual

4.1.5.1: $Lat(j) \leftarrow Lat(j) + Lat(path_t)$

4.1.6: **end for**

4.2: **end for**

4.3: **for** each ***j*** individual in ***P***

4.3.1: **if** $(\sigma_j^2/V_s) > 1$)

$Lscores(j) \leftarrow c_1 * \sigma_j^2/V_s$

else if $((\sigma_j^2/V_s <= 1) AND (Lat(j)/Lat_s <= 1))$

$Lscores(j) \leftarrow 1/c_1 * \sigma_j^2/V_s$

else $Lscores(j) \leftarrow c_2 * (\sigma_j^2/V_s)^m + (1 - c_2) * Lat(j)/Lat_s$

// V_s is the load variance value obtained using Shortest Path

//constants $c_1 = 999$, $c_2 = 0.9$ and $m = 2$ for the given implementation

4.4: **end for**

4.5: Calculate number of $n_{parents}$ needed based on the number of elite kids and crossover/ mutation kids

4.6: **for** q $\leftarrow$ 1 to $n_{parents}$

4.6.1: Randomly select ***k*** individuals from ***P*** and insert the individual having the least L_{scores} value in the set ***Parents***
// ***k*** is the tournament size

4.7: **end for**

4.8: Sort the individuals in the set ***Parents*** in ascending order based on L_{scores} value.

4.9: Select the first N_E values in L_{scores} and the corresponding N_E individuals and insert them in the set ***eliteKids***

4.10: Generate crossover kids (where the number of crossover kids is based on the CrossoverFraction ***F***) from the set ***Parents*** by randomly selecting a crossover point for each pair of parents, performing single point crossover and then putting them in the set ***xoverKids***

4.11: Generate mutation kids, ***mutateKids*** from the set ***xoverKids*** by randomly selecting a gene/flow F_{sd} (mutation point) from an ***xoverKids*** individual followed by randomly selecting an integer value in the range [1, total number paths for the flow F_{sd}] and assigning this value to the gene at the mutation point
// ***s*** corresponds to source and d corresponds to destination, ∀ ***s***, ***d***

4.12: Insert ***eliteKids*** and ***mutateKids*** to the set ***nextPopulation***

4.13: P $\leftarrow$ nextPopulation

4.14: temp $\leftarrow$ 0

4.15: **for** each ***j*** individual in ***P***

4.15.1: temp ← temp + $L_{scores}(j)$
4.16: **end for**
4.17: avg_lscores ← temp/P
4.18: **if** avg_lscores = $L_{scores}(1)$
4.18.1: break
4.19: **endif**
5: **end for**
6: $F_{effective}$ ← the individual from the population $\boldsymbol{P}$ having minimum L_{scores} value
7: **return** $F_{effective}$

Let F_{ij} represent the set of all available paths between S_i and S_j. Considering the that the existing load is concerned with the flows between the pair of hosts connected to (S_1, S_2), (S_2, S_8), (S_3, S_7) and (S_3, S_8). The set of all available paths for $F_{12}, F_{28}, F_{37}, F_{38}$ for the SDN of Fig. 1 is shown in Fig. 2. The time complexity of the algorithm GAEFP is $O(N_G * P * (m!))$ where N_G, P and m corresponds to the number of generations, population size and number of switches respectively.

Encoding of Individuals :One of the problems in applying Genetic Algorithm (GA) is the definition of an encoding scheme that describes the one-to-one mapping between the solution and the individual in the population. The individual in the population (as shown in Fig. 3) has the length of $\boldsymbol{k}$, where k is the total number of flows and the $\boldsymbol{k}$ values/genes of an individual corresponds to the serial number of a path for each flow P_{ij} (as shown in Fig. 2). For each flow P_{ij} the total number of paths from the source $\boldsymbol{i}$ to the destination $\boldsymbol{j}$ may be different (since it is independent of the topology of the network).

Fitness function :To evaluate the individual by measuring its fitness (survival measure) that determines how well an individual from a population solves the given problem. The fitness is generally a real number measuring how close the individual is to the optimal solution. The objective is to search for an individual (for load distribution) that minimizes the load variance value, σj2, ∀ j, keeping the latency in each path from source to destination as less as possible. The fitness function used by algorithm GAEFP (specified in step 4.3.1) is explained as is as follows:

$$\begin{aligned} L_{scores}(j) &= c_1 * \sigma_j^2/V_s, \text{ if } (\sigma_j^2/V_s) > 1 \\ &= 1/c_1 * \sigma_j^2/V_s, \text{ if } ((\sigma_j^2/V_s \leq 1) \text{ AND } (Lat(j)/Lat_s \leq 1)) \\ &= c_2 * (\sigma_j^2/V_s)^m + (1 - c_2) * Lat(j)/Lat_s, otherwise \end{aligned}$$

It is checked if the load variance obtained from the jth individual (σ_j^2) is greater than the load variance obtained from the Shortest Path method (V_s) [i.e if $\sigma_j^2/V_s > 1$] then in order to eliminate this individual/give least priority to it for selection the ratio σ_j^2/V_s is multiplied by a considerably large constant value c_1 (like 999) and assigned to $L_{scores}(j)$. If the load variance obtained from the jth individual (σ_j^2) is lesser than or equal to the load variance obtained from the Shortest Path method (V_s) is less than 1 [i.e if $(\sigma_j^2/V_s) \leq 1$ and the latency value of the jth individual (Lat(j)) is lesser than or equal to the latency value obtained using the Shortest Path [i.e. if $((\sigma_j^2/V_s \leq 1)$ AND $(Lat(j)/Lat_s \leq 1))$] then in order to include this individual/give more priority to it for selection the ratio σ_j^2/V_s is multiplied by a considerably small constant value $1/c_1$ (like 0.001) and assigned to $L_{scores}(j)$. Otherwise, the weighted mean of $(\sigma_j^2/V_s)^m$ and $Lat(j)/Lat_s$ is assigned $L_{scores}(j)$. The value of the constants c_2 (like 0.9) and m (like m = 2) are chosen using trial and error method. After 1500 trials it is found that the values of $c_1 = 999$, $c_2 = 0.9$ and $m = 2$ give a considerably good outcome.

Selection function :At each step, the individuals in the current population are combined to generate the next population. It uses the fitness function specified in step 4.3.1 of the algorithm *GAEFP* to calculate the L_{scores} value of each individual. The individuals that have the least Lscores are chosen in the current population and they are called parents. These parents will again contribute their genes to create the individuals for their next generation.

Elitism :The concept of elitism is applied in the algorithm *GAEFP*. In this concept, few individuals who has the less L_{scores} are kept unchanged in each generation. The number of elite children is N_E.

Crossover function :The algorithm selects a pair of parents based on the crossover fraction is $\boldsymbol{F}$ and combines them to form crossover children. *(xoverKids)*. There are various techniques for the crossover process, like single-point, 2-point, multi-point, and uniform crossovers. Algorithm *GAEFP* in step 4.10 generates crossover kids (where the number of crossover kids is based on the CrossoverFraction $\boldsymbol{F}$) from the set ***Parents*** by randomly selecting a crossover point for each pair of parents, performing single point crossover and then inserting them in the set ***xoverKids***.

Mutation function :The mutation function used in this algorithm selects a gene/flow P_{ij} (mutation point) from an individual randomly. The value of the gene at the mutation point is set to an integer value which is selected randomly from the range [1, total number paths for the flow P_{ij}]. Algorithm *GAEFP* in step 4.11 applies this mutation to every children generated by crossover *(xoverKids)*. A simple example showing mutation and crossover is shown in Fig. 4.

F_{12}

S. no.	Path
1.	1 2
2.	1 4 3 2
3.	1 4 2

F_{21}

S. no.	Path
1.	2 1
2.	2 3 4 1
3.	2 4 1

F_{28}

S. no.	Path
1.	2 3 4 5 6 8
2.	2 4 5 6 8
3.	2 3 4 5 8
4.	2 4 5 8
5.	2 3 4 5 7 8
6.	2 4 5 7 8
7.	2 1 4 5 6 8
8.	2 1 4 5 8
9.	2 1 4 5 7 8

F_{82}

S. no.	Path
1.	8 6 5 4 3 2
2.	8 6 5 4 2
3.	8 5 4 3 2
4.	8 5 4 2
5.	8 7 5 4 3 2
6.	8 7 5 4 2
7.	8 6 5 4 1 2
8.	8 5 4 1 2
9.	8 7 5 4 1 2

F_{37}

S. no.	Path
1.	3 4 5 7
2.	3 4 5 8 7
3.	3 4 5 6 8 7
4.	3 2 4 5 7
5.	3 2 4 5 8 7
6.	3 2 4 5 6 8 7
7.	3 2 1 4 5 8 7
8.	3 2 1 4 5 7
9.	3 2 1 4 5 6 8 7

F_{73}

S. no.	Path
1.	7 5 4 3
2.	7 8 5 4 3
3.	7 8 6 5 4 3
4.	7 5 4 2 3
5.	7 8 5 4 2 3
6.	7 8 6 5 4 2 3
7.	7 8 5 4 1 2 3
8.	7 5 4 1 2 3
9.	7 8 6 5 4 1 2 3

F_{38}

S. no.	Path
1.	3 4 5 6 8
2.	3 2 4 5 6 8
3.	3 4 5 8
4.	3 2 4 5 8
5.	3 4 5 7 8
6.	3 2 4 5 7 8
7.	3 2 1 4 5 6 8
8.	3 2 1 4 5 8
9.	3 2 1 4 5 7 8

F_{83}

S. no.	Path
1.	8 6 5 4 3
2.	8 6 5 4 2 3
3.	8 5 4 3
4.	8 5 4 2 3
5.	8 7 5 4 3
6.	8 7 5 4 2 3
7.	8 6 5 4 1 2 3
8.	8 5 4 1 2 3
9.	8 7 5 4 1 2 3

Fig. 2. The set of all available paths for F12, F28, F37, F38 for the SDN of Fig. 1.

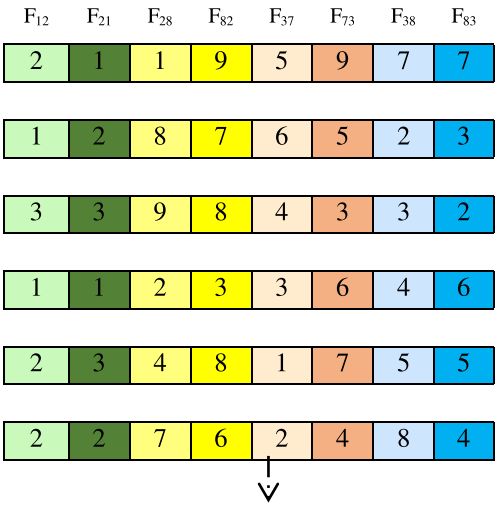


Fig. 3. GA individual mapping for population generation.

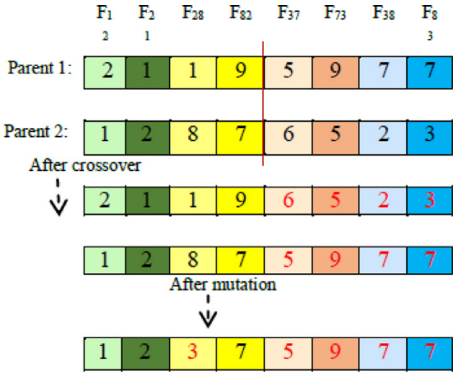


Fig. 4. A simple example showing mutation and crossover.

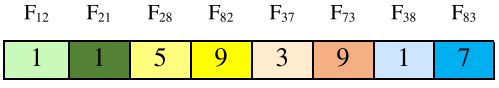


Fig. 5. The sequence obtained using GAEFP.

Considering a homogeneous set of switches and $\beta_i = 0.001$, with a generation value of 100, population size of 50, tournament size of 7, algorithm *GAEFP* generates the individual (shown in Fig. 5) with the load variance value of 0.6875 and latency 0.00975 ms.

The flow paths for this sequence are:

F_{12}: 1 2
F_{21}: 2 1
F_{28}: 2 3 4 5 7 8
F_{82}: 8 7 5 4 1 2
F_{37}: 3 4 5 6 8 7
F_{73}: 7 8 6 5 4 1 2 3
F_{38}: 3 4 5 6 8
F_{83}: 8 6 5 4 1 2 3

Algorithm *Shortest Path* (SP) generates the individual (shown in Fig. 6) with the load variance value of 2.4375 and latency of 0.007 ms.

F_{12}	F_{21}	F_{28}	F_{82}	F_{37}	F_{73}	F_{38}	F_{83}
1	1	2	4	1	1	1	3

Fig. 6. The sequence obtained using Shortest Path.

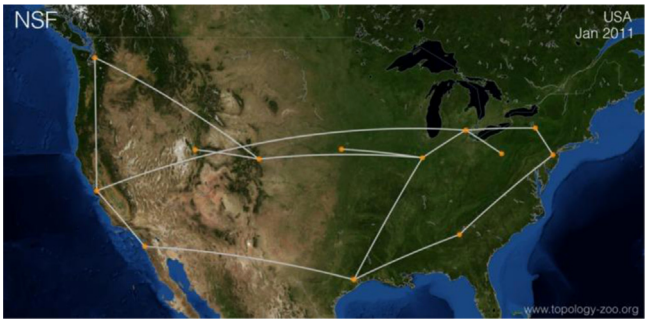


Fig. 7. NSF network obtained from the topology-zoo [52].

The flow paths for this sequence are:
F_{12}: 1 2
F_{21}: 2 1
F_{28}: 2 4 5 6 8
F_{82}: 8 5 4 2
F_{37}: 3 4 5 7
F_{73}: 7 5 4 3
F_{38}: 3 4 5 6 8
F_{83}: 8 5 4 3
Therefore, it is found that using *GAEFP* instead of the shortest path the load variance of the switches got reduced by 1.75 with a negligible delay of 0.00275 ms.

5. A case study using NSF network

The proposed *GAEFP* algorithm is implemented using Python 3.9 running on a PC with Intel(R) Core(TM) i7-8700 CPU @ 3.20 GHz (12 CPUs), ~3.2 GHz, 32.00 GB RAM and Windows 10 OS. The proposed algorithm *TAFR* is applied to the real world online network topology NSF obtained from the Topology Zoo [51,52] and various experiments were performed using the network emulator Mininet. The NSF network comprises of 13 nodes and 15 links as shown in Fig. 7. Using the network emulator Mininet the link latency values are obtained.
The location of switches for this network are as follows:

SEQSUINET,RiceUniversity,Houston ('S1')
JonVonNeumannCenter,Princeton,NJ ('S2')
SURANET,GeorgiaTech,Atlanta ('S3)
PittsburghSupercomputerCenter ('S4')
CornellTheoryCenter,IthacaNY ('S5')
NorthWestNet,Seattle ('S6')
BARRnet,PaloAlto ('S7')
SanDiegoSupercomputerCenter ('S8')
Westnet,SaltLakeCity ('S9')
NCAR,Boulder('S10')
MIDnet,Lincoln,NE ('S11')
NCSA,UniversityofIllinois,Champaign ('S12')
MeritUnivofMichigan,AnnArbor ('S13')

Table 3
Link latency matrix for the NSF network.

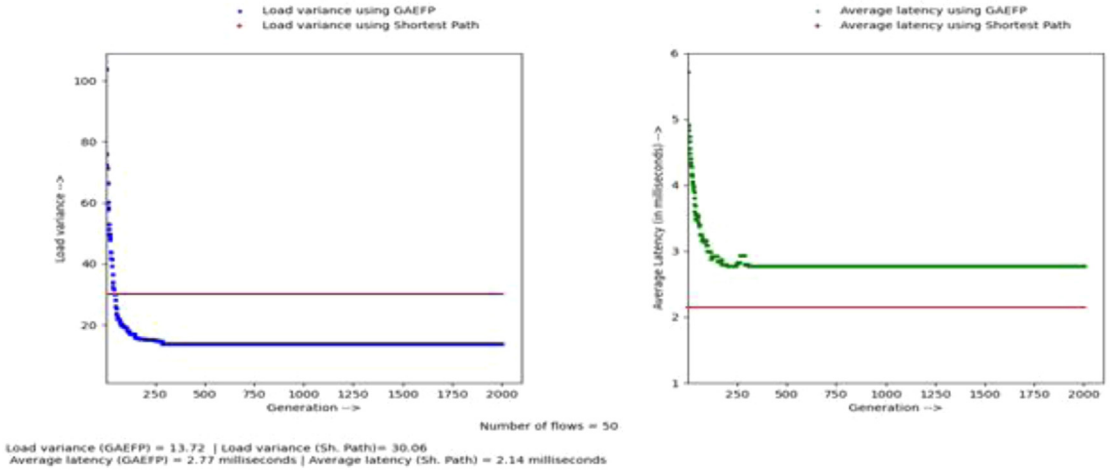
Z	01	02	03	04	05	06	07	08	09	10	11	12	13
01	E	E	1.34	E	E	E	E	1.58	E	E	E	1.27	E
02	E	E	1.02	E	1.16	E	E	E	E	E	E	E	E
03	1.34	1.02	E	E	E	E	E	E	E	E	E	E	E
04	E	E	E	E	E	E	E	E	E	E	E	E	1.18
05	E	1.16	E	E	E	E	E	E	E	E	E	E	0.09
06	E	E	E	E	E	E	1.34	E	E	1.16	E	E	E
07	E	E	E	E	E	1.34	E	0.99	E	E	E	E	0.80
08	1.58	E	E	E	E	E	0.99	E	E	E	E	E	E
09	E	E	E	E	E	E	E	E	E	0.45	E	E	E
10	E	E	E	E	E	1.16	E	E	0.45	E	E	0.68	E
11	E	E	E	E	E	E	E	E	E	E	E	1.11	E
12	1.27	E	E	E	E	E	E	E	E	0.68	1.11	E	1.31
13	E	E	E	1.18	0.09	E	0.80	E	E	E	E	1.31	E

The link latency values obtained from the network topology tools for Mininet are shown in Table 3. Algorithm *GAEFP* is implemented using Python and the plots of lo ad variance and the average latency values of the NSF network for various values of flows generated randomly for 2000 generations of the algorithm are shown in Fig. 8. Since, the traditional *Shortest Path* (SP) algorithm has been used by most of the approaches like Hu et al. [10], Koryachko et al. [13], Wang et al. [53], Lin et al. [15], Chen et al. [16], Raza et al. [17], Rofie et al. [44] (discussed in Section 2), a comparision of the proposed *GAEFP* has been made with traditional *SP* algorithm.

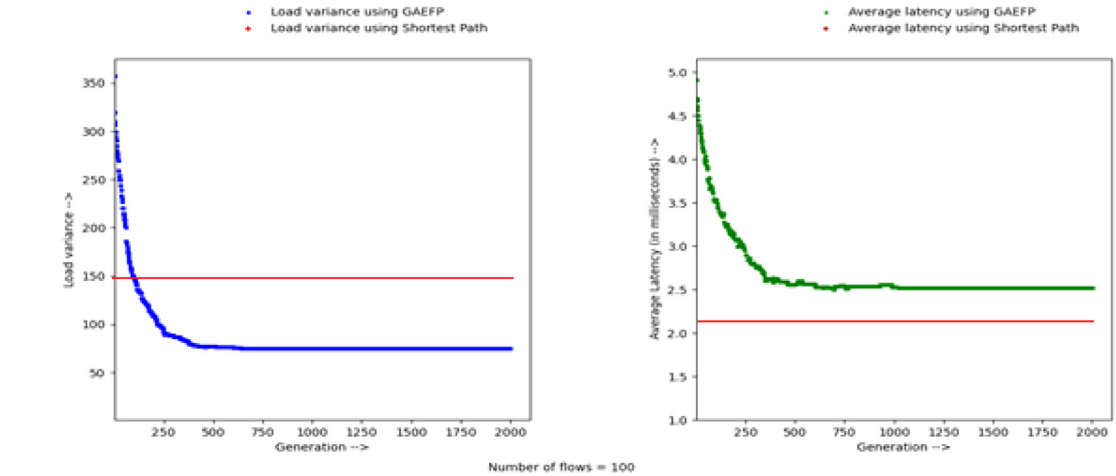
Fig. 8a shows the plots for a set of 50 flows, Fig. 8b shows the plots for a set of 100 flows, Fig. 8c shows the plots for a set of 500 flows. From the plots it is found that algorithm *GAEFP* reached its global minima before 2000 generations. For the instance of execution with 50 flows from source to destination (generated randomly) the load variance value using *GAEFP* is found to be 13.72 with an average latency of 2.77 ms whereas using the traditional *Shortest Path* (SP) algorithm results in the load variance value of 30.06 with an average latency of 2.14 ms (as shown in Fig. 6a). Thus, it is observed that for a negligible difference in the latency values (i.e. 0.63 ms) algorithm *GAEFP* has reduced the load variance values of the switches to less than 50% (i.e. 30.06–13.72 = 16.34). For the instance of execution with 100 flows from source to destination (generated randomly) the load variance value using *GAEFP* is found to be 74.67 with an average latency of 2.52 ms whereas using the traditional *Shortest Pa*th (SP) algorithm results in the load variance value of 149.94 with an average latency of 2.12 ms (as shown in Fig. 8b). Thus, it is observed that for a negligible difference in the latency values (i.e 0.4 ms) algorithm *GAEFP* has reduced the load variance values of the switches to less than 50% (i.e. 149.94 −74.67 = 75.27). For the instance of execution with 500 flows from source to destination (generated randomly) the load variance value using *GAEFP* is found to be 1350.22 with an average latency of 2.45 ms whereas using the traditional Shortest Path (SP) algorithm results in the load variance value of 2693.82 with an average latency of 2.08 ms (as shown in Fig. 6c). Thus, it is again observed that for a negligible difference in the latency values (i.e 0.37 ms) algorithm *GAEFP* has reduced the load variance values of the switches to almost close to 50% (i.e. 2693.82 − 1350.22 = 1343.6). From the plots it is also observed that algorithm *GAEFP* reaches global minima of load variance for various number of flows from various sources to destinations (generated randomly) with a negligible difference in the average latency value (less than 1 ms) with respect to the traditional Shortest Path (SP) algorithm used by most of the approaches like Hu et al. [10], Koryachko et al. [13], Wang et al. [53], Lin et al. [17], Chen et al. [16], Raza et al. [17], Rofie et al. [44] (discussed in Section 2). Although, Mahlab et al. [35] tried to do optimal load distribution using Genetic Algorithm but resulted it in local optima. Cimorelli et al. [40] used Game Theory and Wardrop Equilibrium for load balancing across multiple controllers but it also resulted in local minimum. Thus, algorithm *TAFR* using *GAEFP* resulting in global minima of load variance with a negligible difference in the average latency value (less than 1 ms) with respect to the traditional *Shortest Path* (SP) algorithm used by most of the approaches is found to be quite effective and useful during adverse situations where there is an abnormal increase in the traffic load (in the switches/nodes) due to link failure/node failure, resulting in increase in latency (delay).

6. Performance evaluation

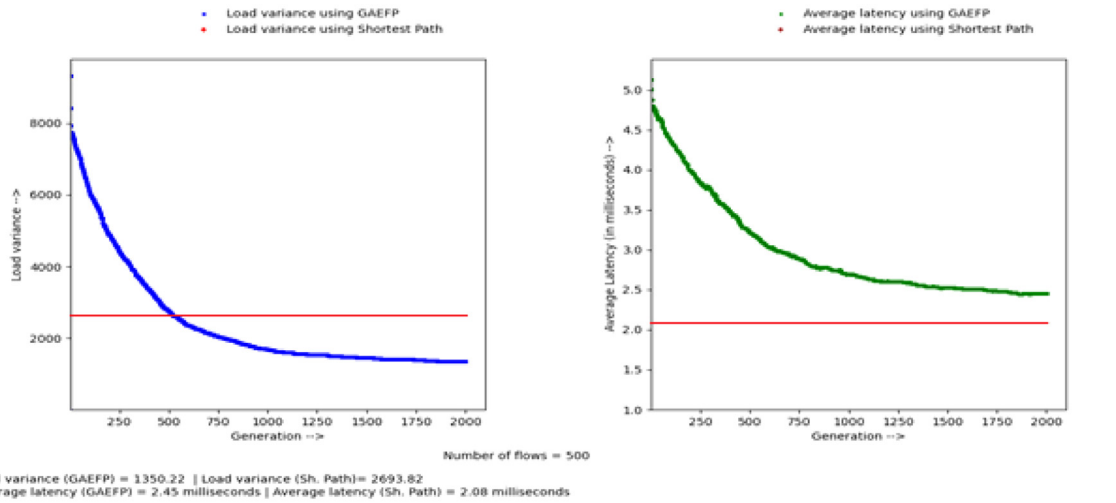
For performance evaluation of the proposed Traffic Aware Flow Routing (TAFR) algorithm the system has been tested in the network emulator Mininet by using simulation based on Monte Carlo simulation for varying number of flows generated randomly from various sources to destinations. A comparative analysis of the proposed approach with other existing approaches have been carried out as shown in Table 4. Since, the traditional Shortest Path (SP) algorithm has been used by most of the approaches like Hu et al. [10], Koryachko et al. [13], Wang et al. [53], Lin et al. [15], Chen et al. [16], Raza et al. [17], Rofie et al. [44] (discussed in Section 2), a comparision of the proposed *GAEFP* has been made with traditional *SP* algorithm for performance evaluation. In order to show the trend in load variance with the increase in the number of flows extensive simulations have been made with various number of flows (generated randomly). Initially, with 50 flows the loads on the switches were less and the load variance among the switches was also less (as shown in Fig. 9a) but as the number of flows increases randomly the load on certain switches are found to be considerably higher



a. Results obtained for 50 flows generated randomly from various sources to destinations



b. Results obtained for 100 flows generated randomly from various sources to destinations



c. Results obtained for 500 flows generated randomly from various sources to destinations

Fig. 8. Plots of load variance and latency values for the NSF network for various values of flows generated randomly for various generations of the algorithm GAEFP.

Table 4
A comparative analysis of various approaches for load distribution in data plane.

Technique	Load distribution
Lee et al. [23]	Used shortest Path. Increased load variance among the switches
Chien et al. [5]	Static load distribution
Xu et al. [7]	Not effective. Suffers from inaccuracy
Hu et al. [10]	Not effective. Increased load variance among the switches. Used All Pair Shortest Path algorithm
Koryachko et al. [13], Wang et al. [14], Lin et al. [15], Chen et al. [16], Raza et al. [17], Rofie et al. [44]	Not effective. Increased load variance among the switches Adopted Shortest Path routes from source to destination
Jiugen et al. [21], Ma et al. [22], Wang et al. [34]	Balances the load among the controllers put in separate regions but not among the switches placed in the same region
Attarha et al. [18], Hwang et al. [24], Ramdhani et al. [25], Boero et al. [26], Chakravarthy and Amutha [27]	Updated flow rules to improve the level of load balancing, but at the cost of increased overhead and latency
Al-Tam and Correia [30]	Developed a heuristic method for switch migration to balance the load using the best controller choice, but it is centralized, has a single point of failure, the migration cost has not been considered, and the bandwidth is underutilized.
Mahlab et al. [35]	Used Genetic Algorithm to enable the optimal localization in the network for the deployment of new links by minimizing the cost without considering the communication latency
Yu et al. [36]	Used Deep Neural Networks with backpropagation to attempt load balancing with efficient path selection, but it used a small number of parameters with added overhead.
Liao et al. [27]	By changing flow table rules their binary tree load balancing method utilized QoS measurements but it is topology dependent and is complicated.
Maity et al. [29]	Presented a load reduction technique that prioritizes traffic flows based on QoS demands but it increased overhead and latency.
Proposed approach	Effective load distribution in the Data Plane with reduced load variance and latency

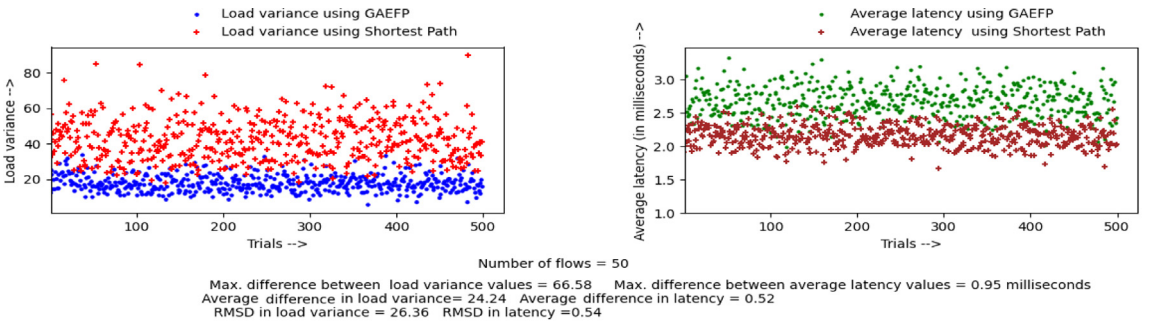
as compared to other switches. Therefore the load variance among the switches is also high (as shown in Fig. 9c). The Root Mean Square Deviation (RMSD) has been calculated using the following formula:

$$RMSD = \frac{\sqrt{\sum_{t=1}^{T}(x_{1,t} - x_{2,t}) * (x_{1,t} - x_{2,t})}}{T} \tag{8}$$

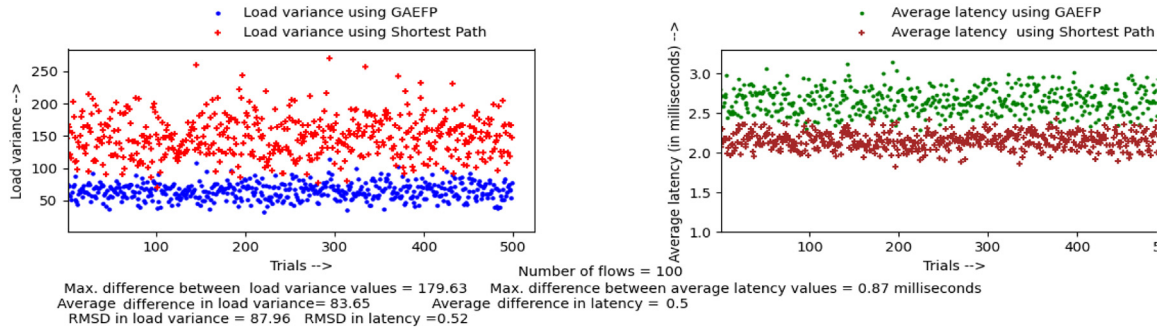
Here $\boldsymbol{T}$ is the total number of trials, $\boldsymbol{t}$ is the trial number, $x_{1,t}$ and $x_{2,t}$ are the obtained values from two different algorithms respectively for each trial $\boldsymbol{t}$. The plots of load variance & latency values for the traditional *SP* algorithm and the proposed *GAEFP* algorithm are shown in Fig. 9.

From Fig. 9, it is found that for 50 flows from various sources and destinations selected randomly for 500 trials it is found that the algorithm *GAEFP* always has a lesser load variance (for the switches) as compared to the *SP* algorithm. The maximum difference between the load variance values is 66.58 and the maximum difference in the average latency values is 0.95 ms. The average difference between the load variance values of the two algorithms is 24.24 and the average difference in the latency values is 0.52 ms. For 50 flows the *RMSD* between the load variance values of the two algorithms is found to be 26.36 and the *RMSD* values for between the average latency values of the two algorithms is found to be 0.54 ms. For 100 flows from various sources and destinations selected randomly for 500 trials it is found that the algorithm *GAEFP* always has a lesser load variance (for the switches) as compared to the *SP* algorithm. The maximum difference between the load variance values is 179.63 and the maximum difference in the average latency values is 0.87 ms. The average difference between the load variance values of the two algorithms is 83.65 and the average difference in the latency values is 0.5 ms. For 100 flows the *RMSD* between the load variance values of the two algorithms is found to be 87.96 and the *RMSD* values for between the average latency values of the two algorithms is found to be 0.52 ms. Table 5 provides the details of the obtained results for 500 trials. From the table it is found that using *GAEFP* instead of SP decreases the load variance of the switches to a considerable extent with a negligible delay (less than 2 ms). Further, a regression analysis using the load variance values obtained from the proposed *GAEFP* and the traditional *Shortest Path* (SP) algorithm is carried out and the results obtained are summarized in Table 6. After the data analysis, it is found that the p-value obtained for the proposed approach is 8.47×10^{-74}.

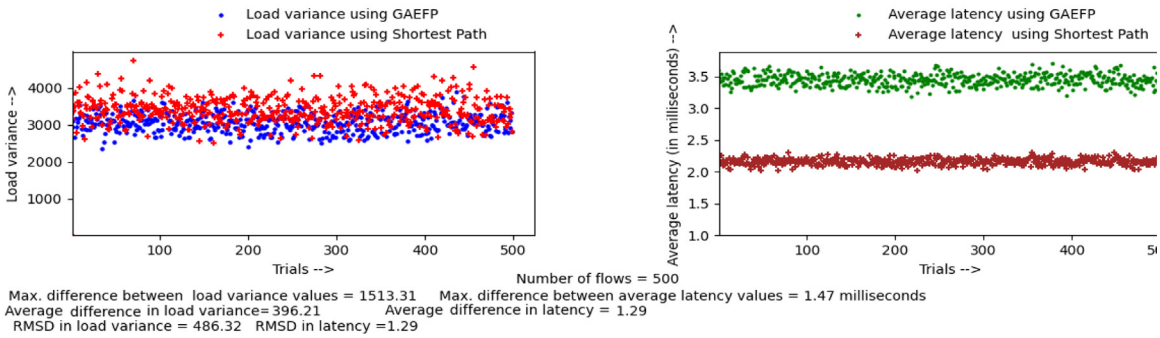
From Table 6, it is found that the regression coefficient $\boldsymbol{r}$ (Multiple R) = 0.69649301 and $p < 0.001$. This indicates that there is a positive relationship between the load variance values obtained from the proposed algorithm and the load



a. Results obtained for 50 flows generated randomly from various sources to destinations



b. Results obtained for 100 flows generated randomly from various sources to destinations



c. Results obtained for 500 flows generated randomly from various sources to destinations

Fig. 9. Plots obtained for the algorithm GAEFP and the traditional SP algorithm for 500 trials.

Table 5
Observations after 500 trials.

Between the algorithms *GAEFP* and *SP*	Number of flows		
	50	100	500
Maximum difference between the load variance values	66.58	179.63	1513.31
Maximum difference in the average latency values	0.95 ms	0.87 ms	1.47 ms
Average difference between the load variance values	24.24	83.65	396.21
Average difference in the latency values	0.52 ms	0.5 ms	1.29 ms
RMSD between the load variance values	26.36	87.96	486.32
RMSD values for between the average latency values	0.54 ms	0.52 ms	1.29 ms

variance values obtained from the *SP* algorithm. Thus, the proposed algorithm using *GAEFP* seems to provide less load variance as compared to the approaches using the traditional *SP* algorithm. Hence, it is found that using *GAEFP* instead of *SP* decreases the load variance of the switches to a considerable extent with a negligible delay (less than 2 ms as shown in Fig. 9). Thus, it can be inferred that algorithm *TAFR* using algorithm *GAEFP* re-routes the packet flows (when the adverse

Table 6
Observations from Regression Analysis.

Regression statistics						
Multiple R	0.69649301					
R Square	0.485102513					
Adjusted R Square	0.484068583					
Standard error	209.6869899					
Observations	500					
ANOVA						
	df	*SS*	*MS*	*F*	*Significance F*	
Regression	1	20 629 327.3	20 629 327	469.1828	8.46731×10^{-74}	
Residual	498	21 896 379.6	43 968.63			
Total	499	42 525 706.9				
	Coefficients	*Standard error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*
Intercept	1635.644271	64.00106546	25.55652	1.16×10^{-92}	1509.898883	1761.389659
Shortest path	0.358874304	0.016568047	21.66063	8.47×10^{-74}	0.326322417	0.391426192

situation arises) in an effective way to avoid abnormal load on some switches/links thereby maintaining an acceptable performance of the network with the existing resources.

7. Conclusion and future work

The components of SDN architecture include the control layer, application layer, and infrastructure layer [54]. The performance of SDN not only depends on the proper choice of controller locations that reduce the communication latency between the switches and the controllers but for a given topology and topography [3], even with the optimal controller and switch positions, SDN is sometimes found to show poor performance causing irritating delays, unreliable data transmission etc. due to the increase in traffic load. The increase in the traffic load can cause congestion resulting in the increase in communication latency. This may manifest as an increase in load in the switches and the controllers. Failure of certain nodes/links can cause an abnormal increase in traffic load at certain switches and controllers resulting in an increase in latency. The existing techniques using the traditional Shortest Path algorithm may not be effective enough to provide good throughput and bandwidth due to the frequent and unexpected nature of network traffic variations causing certain switches to encounter a very high traffic load as compared to others. Under such adverse situations, the performance of the network becomes extremely poor resulting in communication delays and causing annoyance among the users of the network. The existing techniques of load distribution are not always found to be efficient since they either use switch migration techniques, focus on load balancing in the Data Plane using the traditional Shortest Path approach, or focus on load balancing in the Control Plane of SDN. The proposed approach (*TAFR* using *GAEFP*) facilitates traffic-aware artificially intelligent dynamic traffic load distribution in the Data Plane of SDN by re-routing the packet flows (when the adverse situation arises) for effective utilization of the switches/nodes, prevents abnormal differences in traffic load among the switches and maintains an acceptable performance of the network with the existing resources. From the results obtained from the case study, it is found that the proposed approach is quite effective in reducing the abnormal loads on switches/links keeping the latency as less as possible. Further, the proposed approach can be applied for its evaluation with other existing networks. Nevertheless, the proposed approach has been found to be quite effective in avoiding abnormal load on switches/links and maintaining a resilient network with the existing resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgment

The authors would like to acknowledge *School of Computer and Systems Sciences, Jawaharlal Nehru University*, India, for providing necessary support to conduct this research.

References

[1] Y. Zhang, L. Cui, W. Wang, Y. Zhang, A surveyon software defined networking with multiple controllers, J. Netw. Comput. Appl. 103 (2018) 101–118.
[2] M.T.I. Ul Huque, W. Si, G. Jourjon, V. Gramoli, Large-scale dynamic controller placement, IEEE Trans. Netw. Serv. Manag. 14 (1) (2017) 63–76.
[3] Chaitali Dey Bhowmik, Tirthankar Gayen, A technique for topography aware dynamic controller placement in SDN, Int. J. Inf. Technol. (2022) Springer.
[4] H. Tahaei, R. Salleh, S. Khan, R. Izard, K.-K.-R. Choo, N.B. Anuar, A multi-objective software defined network traffic measurement, Measurement 95 (2017) 317–327.
[5] W.-C. Chien, C.-F. Lai, H.-H. Cho, H.-C. Chao, A SDN-sfc based service-oriented load balancing for the IoT applications, J. Netw. Comput. Appl. 114 (2018) 8897.
[6] S. Ejaz, Z. Iqbal, P.A. Shah, B.H. Bukhari, A. Ali, F. Aadil, Traffic load balancing using software defined networking (SDN) controller as virtualized network function, IEEE Access 7 (2019) 46646–46658.
[7] Y. Xu, M. Cello, I.-C. Wang, A. Walid, G. Wilfong, C.H.-P. Wen, M. Marchese, H.J. Chao, Dynamic switch migrationin distributed software-defined networks to achieve controller load balance, IEEE J. Sel. Areas Commun. 37 (3) (2019) 515–529.
[8] H. Jin, G. Yang, B.-Y. Yu, C. Yoo, TALON:Tenant throughput allocation through traffic load-balancing in virtualized software-defined networks, in: The Proceedings of the International Conference on Information Networking, ICOIN, 2019, pp. 233–238.
[9] Y. Gao, Z. Zhang, D. Zhao, Y. Zhang, T. Luo, A hierarchical routing scheme with load balancing in software defined vehicular ad hoc networks, IEEE Access 6 (2018) 73774–73785.
[10] C. Hu, Z. Huang, Z. Lin, An information transport dynamic load balancing policy based on software defined mobile networks, in: Proceedings of International Conference On Information and Computer Technologies, ICICT, 2018, pp. 83–86.
[11] W. Lan, F. Li, X. Liu, Y. Qiu, A dynamic load balancing mechanism for distributed controllers in software-defined networking, in: Proceedings of International Conference on Measuring Technology and Mechatronics Automation, ICMTMA, 2018, pp. 259–262.
[12] Y. Gao, T. Luo, A load balancing schemefor supporting safety applications in heterogeneous software dened LTE-V networks, in: Proc. IEEE 28th Annu. Int. Symp. Pers. Indoor, Mobile Radio Commun., PIMRC, 2017, p. 15.
[13] V. Koryachko, D. Perepelkin, V. Byshov, Approach of dynamic load balancing in software dened networks with QoS, in: Proc. 6th Medit. Conf. Embedded Comput., MECO, 2017, p. 15.
[14] Q. Wang, L. Gao, Y. Yang, J. Zhao, T. Dou, H. Fang, A load balanced algorithm for multi-controller placementin software-defined network, Mechatronic Syst. Control (Formerly Control Intell. Syst. 46 (2) (2018) 72–81.
[15] Y.-D. Lin, C.C. Wang, Y.-J. Lu, Y.-C. Lai, H.-C. Yang, Twotier dynamic load balancing in SDN-enabled Wi-Fi networks, Wirel. Netw. 24 (8) (2018) 2811–2823.
[16] Y. Chen, W. Chen, Y. Hu, L. Zhang, Y. Wei, Dynamic load balancing for software-defined data center networks, in: Proc. International Conference on Collaborative Computing: Networking, Applications and Worksharing CollaborateCom 2016, in: Lecture Notes ofthe Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 201, Springer, Cham, 2017, pp. 286–301.
[17] S.M. Raza, D. Park, Y. Park, K. Lee, H. Choo, Dynamic load balancing of local mobility anchors in software defined networking based proxy mobile IPv6, in: Proceedings of The 10th International Conference on Ubiquitous Information Managementand Communication, 2016, pp. 1–4.
[18] S. Attarha, K.H. Hosseiny, G. Mirjalily, K. Mizanian, A load balanced congestion aware routing mechanism for software defined networks, in: Proceedings of Iranian Conferenceon Electrical Engineering, ICEE, 2017, pp. 2206–2210.
[19] Z. Chen, S. Manzoor, Y. Gao, X. Hei, Achieving load balancing in high-density software defined WiFi networks, in: Proceedings International Conference Frontiers of Information Technology, FIT, 2017, pp. 206–211.
[20] Y. Zhao, C. Liu, H. Wang, X. Fu, Q. Shao, J. Zhang, Load balancing based multi-controller coordinated deployment strategy in software defined optical networks, Opt. Fiber Technol., Mater. Devices Syst. 46 (2018) 198–204.
[21] S. Jiugen, Z. Wei, J. Kunying, X. Ying, Multi-controller deployment algorithm based on load balance in software defined network, J. Electr. Inf. Technol. 40 (2) (2018) 455–461.
[22] Y.-W. Ma, J.-L. Chen, Y.-H. Tsai, K.-H. Cheng, W.-C. Hung, Load balancing multiple controllers mechanism forsoftware-defined networking, Wirel. Personal Commun. 94 (4) (2017) 3549–3574.
[23] Steven Lee, Kuang-Yi Li, Kwan-Yee Chan, Guan-Hao Lai, Yao Chung, Path layout planning and software based FastFailure detection in survivable OpenFlow networks, in: DRCN 2014 –Proceedings of 10th International Conference on Design of ReliableCommunication Networks, 2014.
[24] R.-H. Hwang, H.-P. Tseng, Load balancing and routing mechanism based on software defined network in data centers, in: Proceedings of International Computer Symposium, ICS, 2016, pp. 165–170.
[25] M.F. Ramdhani, S.N. Hertiana, B. Dirgantara, Multipath routing with load balancing and admission control in software-defined networking (SDN), in: Proceedings of 4th InternationalConference of Information and Communication Technology (ICoICT), 2016, pp. 1–6.
[26] L. Boero, M. Cello, C. Garibotto, M. Marchese, M. Mongelli, BeaQoS: Load balancing and dead line management of queues in an OpenFlow SDN switch, Computer. Netw. 106 (2016) 161–170.
[27] V.D. Chakravarthy, B. Amutha, A novelsoftware-defined networking approach for load balancing in data center networks, Int. J. Commun. Syst. (2019) e4213.
[28] M. Priyadarsini, J.C. Mukherjee, P. Bera, S. Kumar, A.H.M. Jakaria, M.A. Rahman, An adaptive load balancing scheme for software defined network controllers, Comput. Netw. 164 (2019) 106918.
[29] A.K. Rangisetti, T.V.S. Pasca, B.R. Tamma, Qos aware load balance in software defined LTE networks, Comput. Commun. 97 (2017) 52–71.
[30] F. Al-Tam, N. Correia, On load balancingvia switch migration in software-defined networking, IEEE Access 7 (2019) 95998–96010.
[31] S. Zhang, J. Lan, P. Sun, Y. Jiang, Onlineload balancing for distributed control plane in software-defined data center network, IEEE Access 6 (2018) 18184–18191.
[32] A. Pietrabissa, L.R. Celsi, F. Cimorelli, V. Suraci, F.D. Priscoli, A. Di Giorgio, A. Giuseppi, S. Monaco, Lyapunov-based design of a distributed wardrop load-balancing algorithm with application to software-defined networking, IEEE Trans. Control Syst. Technol. 27 (5) (2019) 1924–1936.
[33] M. Farhoudi, P. Habibi, M. Sabaei, Serverload balancing in software-defined networks, in: Proceedings Of 9th International Symposium on Telecommunications, IST, 2018, pp. 435–441.
[34] C. Wang, B. Hu, S. Chen, D. Li, B. Liu, A switch migration based decision-making scheme for balancing loadin SDN, IEEE Access 5 (2017) 4537–4544.
[35] U. Mahlab, P.E. Omiyi, H. Hundert, Y. Wolbrum, O. Elimelech, I. Aharon, K.S.Z. Erlich, S. Zarakovsky, Entropy-basedload balancing for software-defined elastic optical networks, in: Proc. 19th International Conference on Transparent Optical Networks, ICTON, 2017, pp. 1–4.
[36] C. Yu, Z. Zhao, Y. Zhou, H. Zhang, Intelligent optimizing scheme for load balancing in software defined networks, in: Proceedings of 85th IEEE Conference on Vehicular Technology, VTC, 2017, pp. 1–5.
[37] F. Cimorelli, F.D. Priscoli, A. Pietrabissa, L.R. Celsi, V. Suraci, L. Zuccaro, A distributed load balancing algorithm for the control plane in software defined networking, in: Proceedings of 24th Mediterranean Conference on Control and Automation, MED, 2016, pp. 1033–1040.

[38] X. He, Z. Ren, C. Shi, J. Fang, A novelload balancing strategy of software-defined cloud/fog networking inthe internet of vehicles, China Commun. 13 (2016) 140–149.
[39] W.-H. Liao, S.-C. Kuai, C.-H. Lu, Dynamicload-balancing mechanism for software-defined networking, in: Proceedings of. International Conference on Networking and Network Applications(NaNA), 2016, pp. 336–341.
[40] S. Sathyanarayana, M. Moh, Joint route-serverload balancing in software defined networks using ant colony optimization, in: Proceedings Nternational Conference on High Performance Computing & Simulation, HPCS, 2016, pp. 156–163.
[41] C.-T. Yang, S.-T. Chen, J.-C. Liu, Y.-W. Su, D. Puthal, R. Ranjan, A predictive load balancing techniquefor software defined networked cloud services, Computing 101 (3) (2019) 211–235, Springer.
[42] S. Manzoor, X. Hei, W. Cheng, A multi-controllerload balancing strategy for software defined WiFi networks, in: X. Sun, Z. Pan, E. Bertino (Eds.), Cloud Computing and Security. ICCCS 2018, in: Lecture Notes in Computer Science, vol. 11066, Springer, Cham, 2018, pp. 622–633.
[43] H. Xue, K. Kim, H. Youn, Dynamic load balancing of software defined networking based on genetic-ant colony optimization, Sensors 19 (2) (2019) 311.
[44] S.A.M. Rofie, I. Ramli, K.N. Redzwan, S.M.M. Hassan, M.S.B. Ibrahim, OpenFlow based load balancingfor software-defined network applications, J. Comput. Theor. Nanosci. 24 (2) (2018) 1210–1213.
[45] I. Maity, S. Misra, C. Mandal, DART:Data plane load reduction for traffic flow migration in SDN, IEEE Trans. Commun. 69 (3) (2021) 1765–1774.
[46] T.R. Gopalakrishnan Nair, Kavitha Sooda, Comparisonof genetic algorithm and simulated annealing technique for OptimalPath selection in network routing, in: Proceedings of AICTE Sponsored National Conference on VLSI and Networks-NCVN-09, India, 2009, pp. 36–41.
[47] Harish Garg, A hybrid GSA-GA algorithm for constrained optimization problems, Inform. Sci. 478 (2019) 499–523, Elsevier.
[48] Y. Fang, J. Li, A review of TournamentSelection in genetic programming, in: Z. Cai, C. Hu, Z. Kang, Y. Liu (Eds.), Advances in Computation and Intelligence. ISICA 2010, in: Lecture Notes in Computer Science, vol. 6382, Springer, Berlin, Heidelberg, 2010, pp. 181–192, (2010).
[49] Y. Lavinas, C. Aranha, T. Sakurai, M. Ladeira, Experimental analysis of the tournament size on GeneticAlgorithms, in: 2018 IEEE International Conference on Systems, Man, and Cybernetics, SMC, 2018, pp. 3647–3653.
[50] T. Blickle, L. Thiele, A mathematical analysisof tournament selection, in: ICGA, 1995, pp. 9–16.
[51] S. Knight, H.X. Nguyen, N. Falkner, R. Bowden, M. Roughan, The internet topology zoo, IEEE J. Sel. Areas Commun. 29 (9) (2011) 1765–1775.
[52] http://www.topology-zoo.org/.
[53] W. Yong, T. Xiaoling, H. Qian, K. Yuwen, Adynamic load balancing method of cloud-center based on SDN, China Commun. 13 (2) (2016) 130–137.
[54] C.D. Bhowmik, T. Gayen, An Effective Technique of Protocol-Aware Network Traffic Classificationfor Performance Enhancement of SDN, in: Lecture Notes in Networks and Systems, vol. 166, Springer, 2021, pp. 547–556.



Chaitali Dey Bhowmik is presently a Ph.D. scholar in School of Computer and System Sciences, Jawaharlal Nehru University, New Delhi, India. She received M.Tech. in Information Technology from West Bengal University of Technology, India in 2012. Her research interest includes Software Defined Network, Network Optimization and Reliable Network Design.



Tirthankar Gayen is presently working as Assistant Professor, in the School of Computer and System Sciences, Jawaharlal Nehru University, New Delhi, India. He received his Ph.D. from IIT Kharagpur and M.Tech. from IIEST Shibpur, India. He worked as Senior Research Associate (postdoctoral) at City University of Hong Kong and Associate Scientist at ABB. He served as a reviewer of various International journals of IEEE, Springer, Elsevier, Taylor & Francis, etc. His area of interest includes Software Engineering, System Reliability Assessment & Performance Enhancement, Software Defined Network, Natural Language Processing and Cloud Computing.